

SchrotbergerDSC680_Project2

August 16, 2026

0.1 DSC 680 Week Project 2

0.1.1 What Influences Housing Prices?

Caitie Schrotberger
Bellevue University
DSC680-T301 Applied Data Science (2267-1)
Sue Ashton
August 16, 2026

```
[1]: # Import packages needed for analysis
import numpy as np
import pandas as pd
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import mean_absolute_error
```

```
[2]: # Import CSV to dataframe
housing_data=pd.read_csv(r'G:/My Drive/College/Caitie/DSC680/Project2/
↳redfin_single_family_residential_2016_2026.csv')
mortgage=pd.read_csv(r'G:/My Drive/College/Caitie/DSC680/Project2/
↳MORTGAGE30USMONTHLY.csv')

# Display first 5 rows of data
housing_data.head()
```

```
[2]:  LAST UPDATED  FREQUENCY  PERIOD BEGIN  PERIOD END  REGION ID  REGION TYPE  \
0      8/3/2026    Monthly    1/1/2016   1/31/2016      NaN      Country
1      8/3/2026    Monthly    2/1/2016   2/29/2016      NaN      Country
2      8/3/2026    Monthly    3/1/2016   3/31/2016      NaN      Country
3      8/3/2026    Monthly    4/1/2016   4/30/2016      NaN      Country
4      8/3/2026    Monthly    5/1/2016   5/31/2016      NaN      Country
```

	REGION NAME	PROPERTY TYPE	IS SEASONALLY ADJUSTED	HOMES SOLD \
0	National	Single Family Residential	True	269912
1	National	Single Family Residential	True	256233
2	National	Single Family Residential	True	255686
3	National	Single Family Residential	True	269877
4	National	Single Family Residential	True	273174

	... MEDIAN NEW LISTING PRICE (\$)	MEDIAN NEW LISTING PRICE YOY (%) \
0	232434	6.72
1	233151	5.11
2	235964	7.07
3	234593	5.65
4	235890	5.44

	MEDIAN NEW LISTING PRICE PER SQ.FT. (\$) \
0	117.61
1	118.04
2	118.55
3	118.87
4	119.08

	MEDIAN NEW LISTING PRICE PER SQ.FT. YOY (%) \
0	6.17
1	5.51
2	5.40
3	5.25
4	4.98

	MEDIAN SALE PRICE PER SQ.FT. (\$) MEDIAN SALE PRICE PER SQ.FT. YOY (%) \
0	111.80 5.55
1	111.52 4.51
2	111.91 4.34
3	112.63 4.79
4	113.03 4.40

	MONTHS OF SUPPLY MONTHS OF SUPPLY YOY (%) \
0	4.4 -88.82
1	4.6 -39.96
2	4.6 -37.72
3	4.4 -55.77
4	4.2 -62.74

	PERCENT OFF MARKET IN TWO WEEKS (%) \
0	20.37
1	21.26
2	21.87

```

3                22.32
4                22.37

    PERCENT OFF MARKET IN TWO WEEKS YOY (PPTS)
0                4.25
1                4.27
2                4.86
3                3.84
4                3.40

```

[5 rows x 37 columns]

```
[3]: mortgage.head()
```

```

[3]:  observation_date  MORTGAGE30US
0      1/31/2016      3.79
1      2/29/2016      3.62
2      3/31/2016      3.71
3      4/30/2016      3.66
4      5/31/2016      3.64

```

```
[4]: housing_data.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 127 entries, 0 to 126
Data columns (total 37 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   LAST UPDATED                         127 non-null    str
1   FREQUENCY                           127 non-null    str
2   PERIOD BEGIN                         127 non-null    str
3   PERIOD END                           127 non-null    str
4   REGION ID                           0 non-null      float64
5   REGION TYPE                         127 non-null    str
6   REGION NAME                         127 non-null    str
7   PROPERTY TYPE                       127 non-null    str
8   IS SEASONALLY ADJUSTED              127 non-null    bool
9   HOMES SOLD                         127 non-null    int64
10  HOMES SOLD YOY (%)                 127 non-null    float64
11  MEDIAN SALE PRICE NSA ($)           127 non-null    int64
12  MEDIAN SALE PRICE NSA YOY (%)       127 non-null    float64
13  MEDIAN DAYS ON MARKET (DAYS)       127 non-null    int64
14  MEDIAN DAYS ON MARKET YOY (%)      127 non-null    int64
15  AVERAGE SALE TO LIST RATIO (%)     127 non-null    float64
16  AVERAGE SALE TO LIST RATIO YOY (PPTS) 127 non-null    float64
17  SHARE SOLD ABOVE ORIGINAL LIST (%)  127 non-null    float64
18  SHARE SOLD ABOVE ORIGINAL LIST YOY (PPTS) 127 non-null    float64
19  NEW LISTINGS                       127 non-null    int64

```

```

20 NEW LISTINGS YOY (%)          127 non-null    float64
21 ACTIVE LISTINGS              127 non-null    int64
22 ACTIVE LISTINGS YOY (%)      127 non-null    float64
23 INVENTORY                    127 non-null    int64
24 INVENTORY YOY (%)            127 non-null    float64
25 PENDING SALES                127 non-null    int64
26 PENDING SALES YOY (%)        127 non-null    float64
27 MEDIAN NEW LISTING PRICE ($)  127 non-null    int64
28 MEDIAN NEW LISTING PRICE YOY (%) 127 non-null    float64
29 MEDIAN NEW LISTING PRICE PER SQ.FT. ($) 127 non-null    float64
30 MEDIAN NEW LISTING PRICE PER SQ.FT. YOY (%) 127 non-null    float64
31 MEDIAN SALE PRICE PER SQ.FT. ($) 127 non-null    float64
32 MEDIAN SALE PRICE PER SQ.FT. YOY (%) 127 non-null    float64
33 MONTHS OF SUPPLY             127 non-null    float64
34 MONTHS OF SUPPLY YOY (%)     127 non-null    float64
35 PERCENT OFF MARKET IN TWO WEEKS (%) 127 non-null    float64
36 PERCENT OFF MARKET IN TWO WEEKS YOY (PPTS) 127 non-null    float64
dtypes: bool(1), float64(20), int64(9), str(7)
memory usage: 45.0 KB

```

```

[5]: housing_data = housing_data.drop(columns=['LAST UPDATED', 'FREQUENCY', 'PERIOD_
↳BEGIN', 'REGION ID', 'REGION TYPE', 'REGION NAME',
                                         'PROPERTY TYPE', 'IS SEASONALLY_
↳ADJUSTED'])

```

```

[6]: housing_data['PERIOD END'] = pd.to_datetime(housing_data['PERIOD END'])
mortgage['observation_date'] = pd.to_datetime(mortgage['observation_date'])

housing_data = pd.merge(housing_data, mortgage, left_on='PERIOD END',
↳right_on='observation_date', how='inner')
housing_data = housing_data.drop(housing_data.columns[[29]], axis=1)
housing_data.info()

```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 127 entries, 0 to 126
```

```
Data columns (total 30 columns):
```

#	Column	Non-Null Count	Dtype
0	PERIOD END	127 non-null	datetime64[us]
1	HOMES SOLD	127 non-null	int64
2	HOMES SOLD YOY (%)	127 non-null	float64
3	MEDIAN SALE PRICE NSA (\$)	127 non-null	int64
4	MEDIAN SALE PRICE NSA YOY (%)	127 non-null	float64
5	MEDIAN DAYS ON MARKET (DAYS)	127 non-null	int64
6	MEDIAN DAYS ON MARKET YOY (%)	127 non-null	int64
7	AVERAGE SALE TO LIST RATIO (%)	127 non-null	float64
8	AVERAGE SALE TO LIST RATIO YOY (PPTS)	127 non-null	float64
9	SHARE SOLD ABOVE ORIGINAL LIST (%)	127 non-null	float64

10	SHARE SOLD ABOVE ORIGINAL LIST YOY (PPTS)	127 non-null	float64
11	NEW LISTINGS	127 non-null	int64
12	NEW LISTINGS YOY (%)	127 non-null	float64
13	ACTIVE LISTINGS	127 non-null	int64
14	ACTIVE LISTINGS YOY (%)	127 non-null	float64
15	INVENTORY	127 non-null	int64
16	INVENTORY YOY (%)	127 non-null	float64
17	PENDING SALES	127 non-null	int64
18	PENDING SALES YOY (%)	127 non-null	float64
19	MEDIAN NEW LISTING PRICE (\$)	127 non-null	int64
20	MEDIAN NEW LISTING PRICE YOY (%)	127 non-null	float64
21	MEDIAN NEW LISTING PRICE PER SQ.FT. (\$)	127 non-null	float64
22	MEDIAN NEW LISTING PRICE PER SQ.FT. YOY (%)	127 non-null	float64
23	MEDIAN SALE PRICE PER SQ.FT. (\$)	127 non-null	float64
24	MEDIAN SALE PRICE PER SQ.FT. YOY (%)	127 non-null	float64
25	MONTHS OF SUPPLY	127 non-null	float64
26	MONTHS OF SUPPLY YOY (%)	127 non-null	float64
27	PERCENT OFF MARKET IN TWO WEEKS (%)	127 non-null	float64
28	PERCENT OFF MARKET IN TWO WEEKS YOY (PPTS)	127 non-null	float64
29	MORTGAGE30US	127 non-null	float64

dtypes: datetime64[us](1), float64(20), int64(9)
memory usage: 29.9 KB

```
[7]: housing_data.head()
```

```
[7]:
```

	PERIOD END	HOMES SOLD	HOMES SOLD YOY (%)	MEDIAN SALE PRICE NSA (\$)	\
0	2016-01-31	269912	14.75	205900	
1	2016-02-29	256233	6.62	203000	
2	2016-03-31	255686	3.01	212100	
3	2016-04-30	269877	8.94	219000	
4	2016-05-31	273174	7.12	225000	

	MEDIAN SALE PRICE NSA YOY (%)	MEDIAN DAYS ON MARKET (DAYS)	\
0	8.37	69	
1	4.64	69	
2	5.00	68	
3	4.78	66	
4	3.69	66	

	MEDIAN DAYS ON MARKET YOY (%)	AVERAGE SALE TO LIST RATIO (%)	\
0	-900	97.50	
1	-800	97.46	
2	-800	97.45	
3	-800	97.48	
4	-600	97.50	

	AVERAGE SALE TO LIST RATIO YOY (PPTS)	SHARE SOLD ABOVE ORIGINAL LIST (%)	\
--	---------------------------------------	------------------------------------	---

0	0.43	18.12
1	0.33	18.25
2	0.26	18.70
3	0.27	19.09
4	0.27	19.33

	... MEDIAN NEW LISTING PRICE YOY (%) \
0	6.72
1	5.11
2	7.07
3	5.65
4	5.44

	MEDIAN NEW LISTING PRICE PER SQ.FT. (\$) \
0	117.61
1	118.04
2	118.55
3	118.87
4	119.08

	MEDIAN NEW LISTING PRICE PER SQ.FT. YOY (%) \
0	6.17
1	5.51
2	5.40
3	5.25
4	4.98

	MEDIAN SALE PRICE PER SQ.FT. (\$) MEDIAN SALE PRICE PER SQ.FT. YOY (%) \
0	111.80 5.55
1	111.52 4.51
2	111.91 4.34
3	112.63 4.79
4	113.03 4.40

	MONTHS OF SUPPLY MONTHS OF SUPPLY YOY (%) \
0	4.4 -88.82
1	4.6 -39.96
2	4.6 -37.72
3	4.4 -55.77
4	4.2 -62.74

	PERCENT OFF MARKET IN TWO WEEKS (%) \
0	20.37
1	21.26
2	21.87
3	22.32
4	22.37

	PERCENT OFF MARKET IN TWO WEEKS YOY (PPTS)	MORTGAGE30US
0	4.25	3.79
1	4.27	3.62
2	4.86	3.71
3	3.84	3.66
4	3.40	3.64

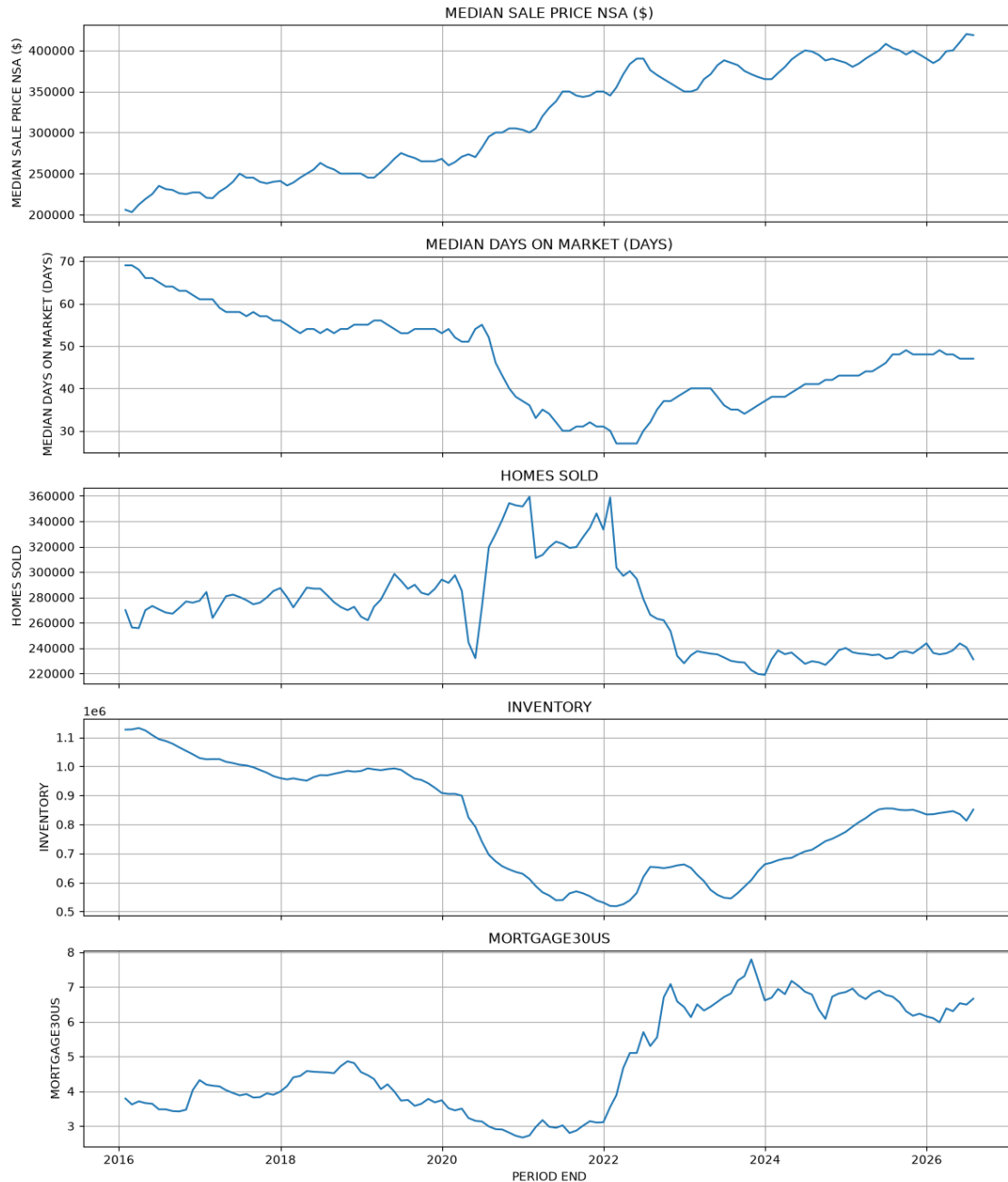
[5 rows x 30 columns]

```
[8]: # Small Multiples Time Series Review
metrics = [
    "MEDIAN SALE PRICE NSA ($)",
    "MEDIAN DAYS ON MARKET (DAYS)",
    "HOMES SOLD",
    "INVENTORY",
    "MORTGAGE30US"
]

fig, axes = plt.subplots(len(metrics), 1, figsize=(12, 14), sharex=True)

for ax, col in zip(axes, metrics):
    sns.lineplot(data=housing_data, x="PERIOD END", y=col, ax=ax)
    ax.set_title(col)
    ax.grid(True)

plt.tight_layout()
plt.show()
```



[9]: *#Market Balance - Days on Market vs Months of Supply*

```
fig, ax1 = plt.subplots(figsize=(12,6))
```

```
sns.lineplot(data=housing_data, x="PERIOD END", y="MEDIAN DAYS ON MARKET_↵
↵(DAYS)", ax=ax1, color="tab:blue")
```

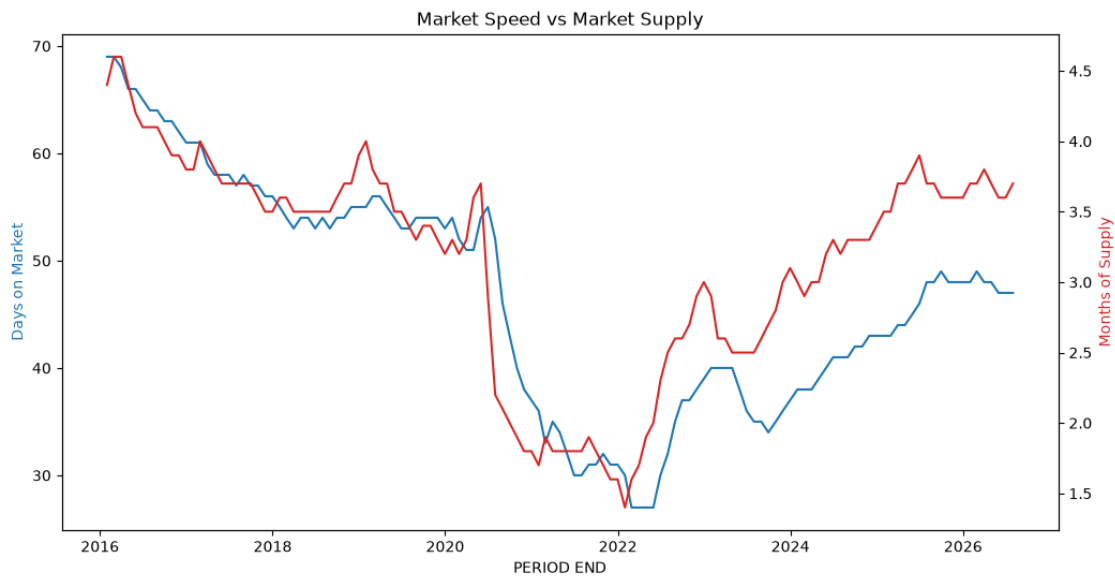
```
ax1.set_ylabel("Days on Market", color="tab:blue")
```

```

ax2 = ax1.twinx()
sns.lineplot(data=housing_data, x="PERIOD END", y="MONTHS OF SUPPLY", ax=ax2,
             color="tab:red")
ax2.set_ylabel("Months of Supply", color="tab:red")

plt.title("Market Speed vs Market Supply")
plt.show()

```

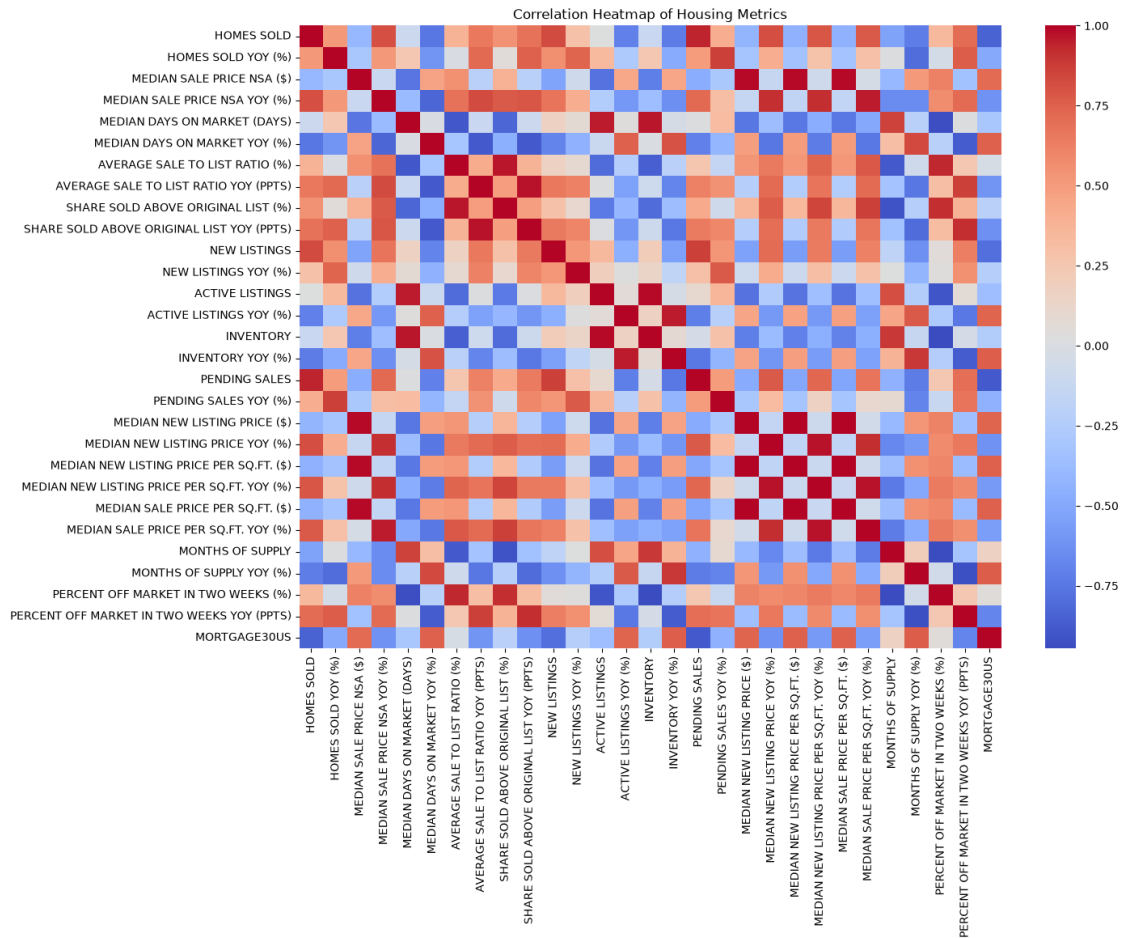


```

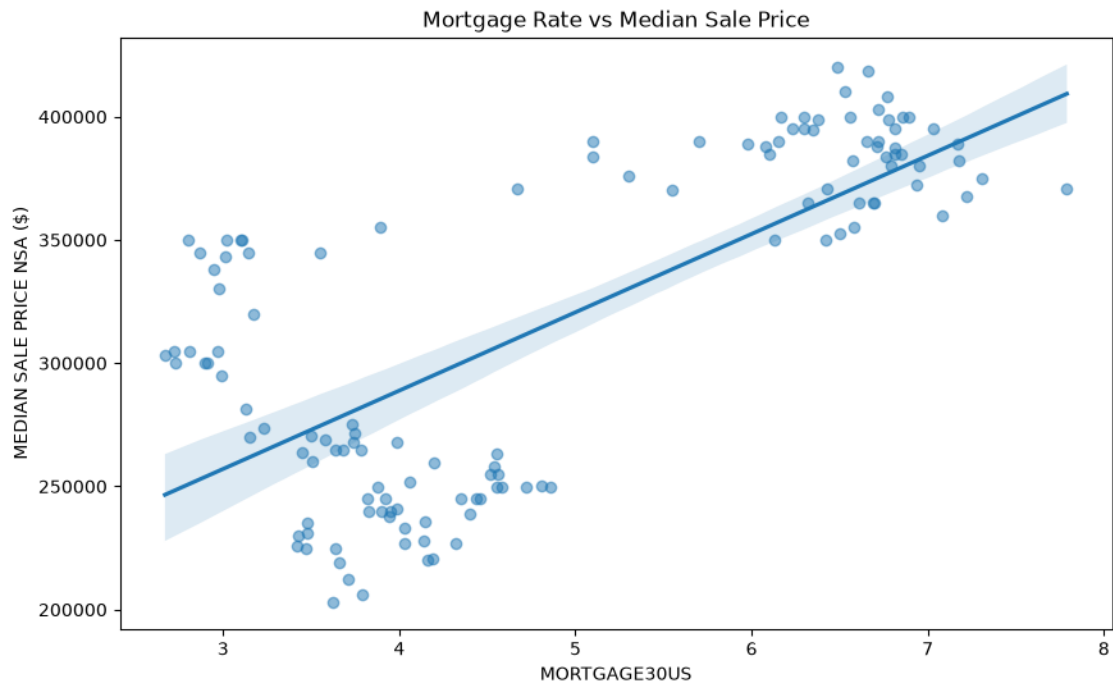
[10]: # Correlation Heatmap
plt.figure(figsize=(14,10))
corr = housing_data.corr(numeric_only=True)

sns.heatmap(corr, cmap="coolwarm", annot=False)
plt.title("Correlation Heatmap of Housing Metrics")
plt.show()

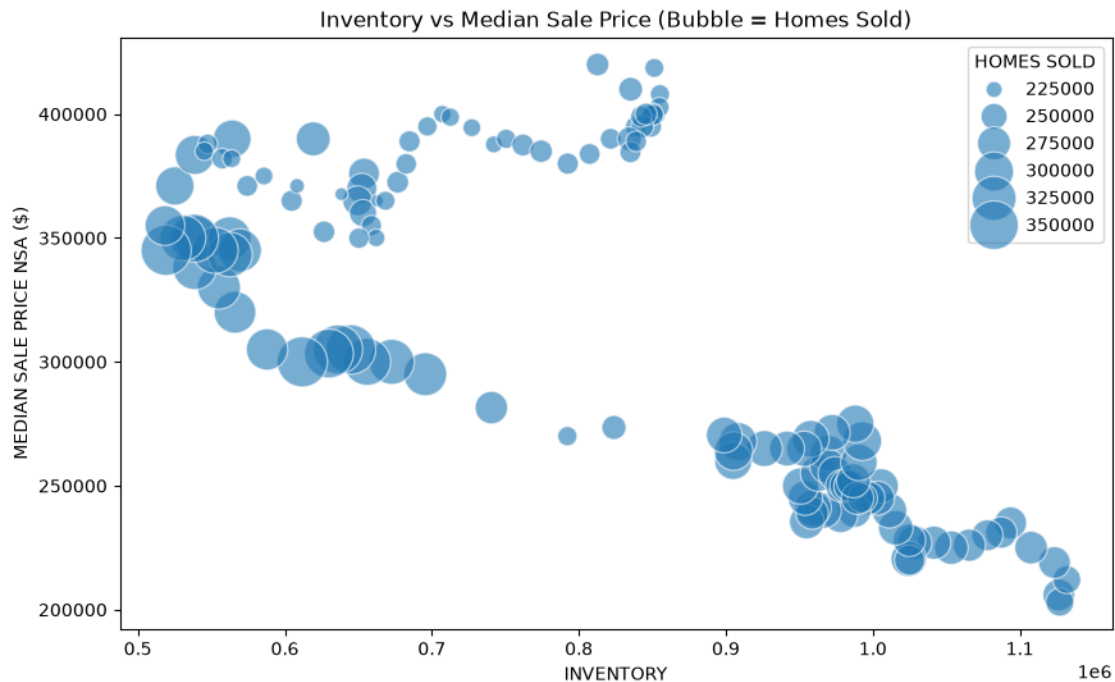
```



```
[11]: # Mortgage Rate vs Median Sale Price Scatter Plot
plt.figure(figsize=(10,6))
sns.regplot(
    data=housing_data,
    x="MORTGAGE30US",
    y="MEDIAN SALE PRICE NSA ($)",
    scatter_kws={"alpha":0.5}
)
plt.title("Mortgage Rate vs Median Sale Price")
plt.show()
```

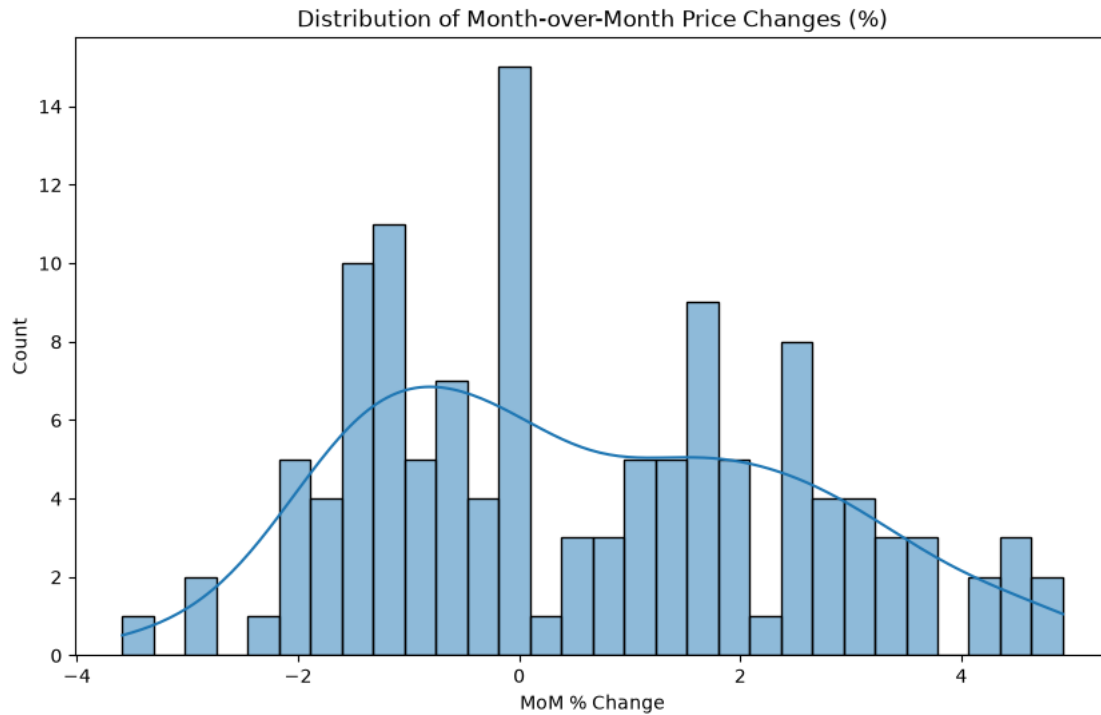


```
[12]: # Inventory vs. Median Sale Price Bubble Chart
plt.figure(figsize=(10,6))
sns.scatterplot(
    data=housing_data,
    x="INVENTORY",
    y="MEDIAN SALE PRICE NSA ($)",
    size="HOMES SOLD",
    sizes=(50, 800),
    alpha=0.6
)
plt.title("Inventory vs Median Sale Price (Bubble = Homes Sold)")
plt.show()
```



```
[13]: # Histogram Monthly Price Changes
housing_data["PRICE_MOM_PCT"] = housing_data["MEDIAN SALE PRICE NSA ($)"]
    .pct_change() * 100

plt.figure(figsize=(10,6))
sns.histplot(housing_data["PRICE_MOM_PCT"].dropna(), kde=True, bins=30)
plt.title("Distribution of Month-over-Month Price Changes (%)")
plt.xlabel("MoM % Change")
plt.show()
```



0.1.2 Logistic Regression

```
[14]: # Create target variable
target = "MEDIAN SALE PRICE NSA ($)"
```

```
[15]: # Create data frame without fields model can't evaluate
keep_cols = [
    'HOMES SOLD',
    'MEDIAN DAYS ON MARKET (DAYS)',
    'AVERAGE SALE TO LIST RATIO (%)',
    'SHARE SOLD ABOVE ORIGINAL LIST (%)',
    'NEW LISTINGS',
    'ACTIVE LISTINGS',
    'INVENTORY',
    'PENDING SALES',
    'MEDIAN NEW LISTING PRICE ($)',
    'MEDIAN NEW LISTING PRICE PER SQ.FT. ($)',
    'MEDIAN SALE PRICE PER SQ.FT. ($)',
    'MONTHS OF SUPPLY',
    'PERCENT OFF MARKET IN TWO WEEKS (%)',
    'MORTGAGE30US']

X = housing_data[keep_cols].copy()
```

```
y = housing_data[target].copy()
```

```
[16]: #Split train and test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)
```

```
[17]: # Define scaler and scale continous variables
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
[18]: # Linear Regression Model
lin_model = LinearRegression()
lin_model.fit(X_train_scaled, y_train)

lin_pred = lin_model.predict(X_test_scaled)

lin_r2 = r2_score(y_test, lin_pred)
lin_rmse = mean_squared_error(y_test, lin_pred)

print("Linear Regression R2:", lin_r2)
print("Linear Regression RMSE:", lin_rmse)
```

Linear Regression R²: 0.984598089635387
Linear Regression RMSE: 73208126.1495761

```
[19]: mae = mean_absolute_error(y_test, lin_pred)
mape = (mae / y_test.mean()) * 100

print("MAE:", mae)
print("MAPE:", mape)
```

MAE: 6882.598531361387
MAPE: 2.1712955526649913

```
[20]: coef = lin_model.coef_
importance = pd.Series(coef, index=keep_cols).sort_values(key=abs,
↳ascending=False)

print("Linear Regression Feature Importance (by absolute coefficient):")
print(importance)
```

Linear Regression Feature Importance (by absolute coefficient):

MEDIAN SALE PRICE PER SQ.FT. (\$)	131927.332580
MEDIAN NEW LISTING PRICE PER SQ.FT. (\$)	-87186.332197
MEDIAN NEW LISTING PRICE (\$)	20388.534721
ACTIVE LISTINGS	15370.680201
INVENTORY	-13298.667846
HOMES SOLD	-7615.948178

PENDING SALES	7615.574265
MEDIAN DAYS ON MARKET (DAYS)	-6032.469786
PERCENT OFF MARKET IN TWO WEEKS (%)	3211.510926
MORTGAGE30US	-2645.257413
AVERAGE SALE TO LIST RATIO (%)	-2398.329337
NEW LISTINGS	-1555.633931
SHARE SOLD ABOVE ORIGINAL LIST (%)	-1484.962764
MONTHS OF SUPPLY	1302.083816

dtype: float64

```
[21]: # Random Forest Model
rf_model = RandomForestRegressor(
    n_estimators=500,
    random_state=42,
    min_samples_leaf=2
)
rf_model.fit(X_train, y_train)

rf_pred = rf_model.predict(X_test)

rf_r2 = r2_score(y_test, rf_pred)
rf_rmse = mean_squared_error(y_test, rf_pred)

print("Random Forest R²:", rf_r2)
print("Random Forest RMSE:", rf_rmse)
```

Random Forest R²: 0.987346287406518
Random Forest RMSE: 60145434.29187058

```
[22]: importances = pd.Series(
    rf_model.feature_importances_,
    index=keep_cols
).sort_values(ascending=False)

print("\nRandom Forest Feature Importance:")
print(importances)
```

Random Forest Feature Importance:	
MEDIAN SALE PRICE PER SQ.FT. (\$)	0.344872
MEDIAN NEW LISTING PRICE PER SQ.FT. (\$)	0.328263
MEDIAN NEW LISTING PRICE (\$)	0.297180
INVENTORY	0.005157
ACTIVE LISTINGS	0.004578
AVERAGE SALE TO LIST RATIO (%)	0.004513
MORTGAGE30US	0.002934
MEDIAN DAYS ON MARKET (DAYS)	0.002240
MONTHS OF SUPPLY	0.002234
PERCENT OFF MARKET IN TWO WEEKS (%)	0.002075

```
HOMES SOLD                                0.001925
NEW LISTINGS                              0.001457
SHARE SOLD ABOVE ORIGINAL LIST (%)         0.001325
PENDING SALES                             0.001246
dtype: float64
```

0.1.3 Exploratory Data Analysis: Second File

```
[23]: # Import CSV to dataframe
housing_features=pd.read_csv(r'G:/My Drive/College/Caitie/DSC680/Project2/
↳housing.csv')

# Display first 5 rows of data
housing_features.head()
```

```
[23]:
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	\
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	

	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	no	yes	2	yes	furnished
1	no	yes	3	no	furnished
2	no	no	2	yes	semi-furnished
3	no	yes	3	yes	furnished
4	no	yes	2	no	furnished

```
[24]: yes_no_cols = [
        'mainroad',
        'guestroom',
        'basement',
        'hotwaterheating',
        'airconditioning',
        'prefarea'
    ]
housing_features[yes_no_cols] = housing_features[yes_no_cols].replace({'yes': 1,
↳ 'no': 0})
```

```
[25]: housing_features = housing_features.drop(housing_features.columns[[12]], axis=1)
```

```
[26]: housing_features.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
#   ...
```

```

---  -----
0  price          545 non-null  int64
1  area           545 non-null  int64
2  bedrooms       545 non-null  int64
3  bathrooms      545 non-null  int64
4  stories        545 non-null  int64
5  mainroad       545 non-null  object
6  guestroom      545 non-null  object
7  basement       545 non-null  object
8  hotwaterheating 545 non-null  object
9  airconditioning 545 non-null  object
10 parking        545 non-null  int64
11 prefarea       545 non-null  object
dtypes: int64(6), object(6)
memory usage: 51.2+ KB

```

```
[27]: housing_features.isna().sum()
```

```

[27]: price          0
      area          0
      bedrooms      0
      bathrooms     0
      stories       0
      mainroad      0
      guestroom     0
      basement      0
      hotwaterheating 0
      airconditioning 0
      parking       0
      prefarea      0
      dtype: int64

```

```
[28]: housing_features.describe(include='all')
```

```

[28]:
count    price          area    bedrooms    bathrooms    stories \
unique         NaN         NaN         NaN         NaN         NaN
top          NaN         NaN         NaN         NaN         NaN
freq         NaN         NaN         NaN         NaN         NaN
mean    4.766729e+06    5150.541284    2.965138    1.286239    1.805505
std     1.870440e+06    2170.141023    0.738064    0.502470    0.867492
min     1.750000e+06    1650.000000    1.000000    1.000000    1.000000
25%     3.430000e+06    3600.000000    2.000000    1.000000    1.000000
50%     4.340000e+06    4600.000000    3.000000    1.000000    2.000000
75%     5.740000e+06    6360.000000    3.000000    2.000000    2.000000
max     1.330000e+07    16200.000000    6.000000    4.000000    4.000000

mainroad  guestroom  basement  hotwaterheating  airconditioning \

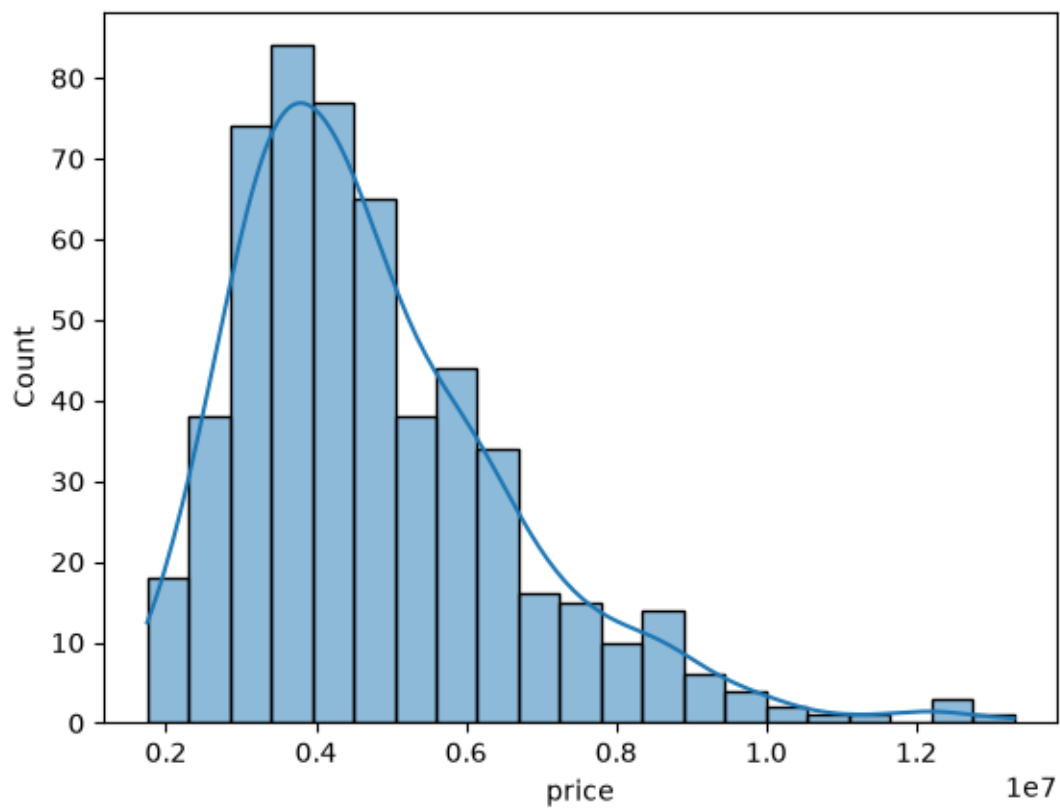
```

count	545.0	545.0	545.0	545.0	545.0
unique	2.0	2.0	2.0	2.0	2.0
top	1.0	0.0	0.0	0.0	0.0
freq	468.0	448.0	354.0	520.0	373.0
mean	NaN	NaN	NaN	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN
min	NaN	NaN	NaN	NaN	NaN
25%	NaN	NaN	NaN	NaN	NaN
50%	NaN	NaN	NaN	NaN	NaN
75%	NaN	NaN	NaN	NaN	NaN
max	NaN	NaN	NaN	NaN	NaN

	parking	prefarea
count	545.000000	545.0
unique	NaN	2.0
top	NaN	0.0
freq	NaN	417.0
mean	0.693578	NaN
std	0.861586	NaN
min	0.000000	NaN
25%	0.000000	NaN
50%	0.000000	NaN
75%	1.000000	NaN
max	3.000000	NaN

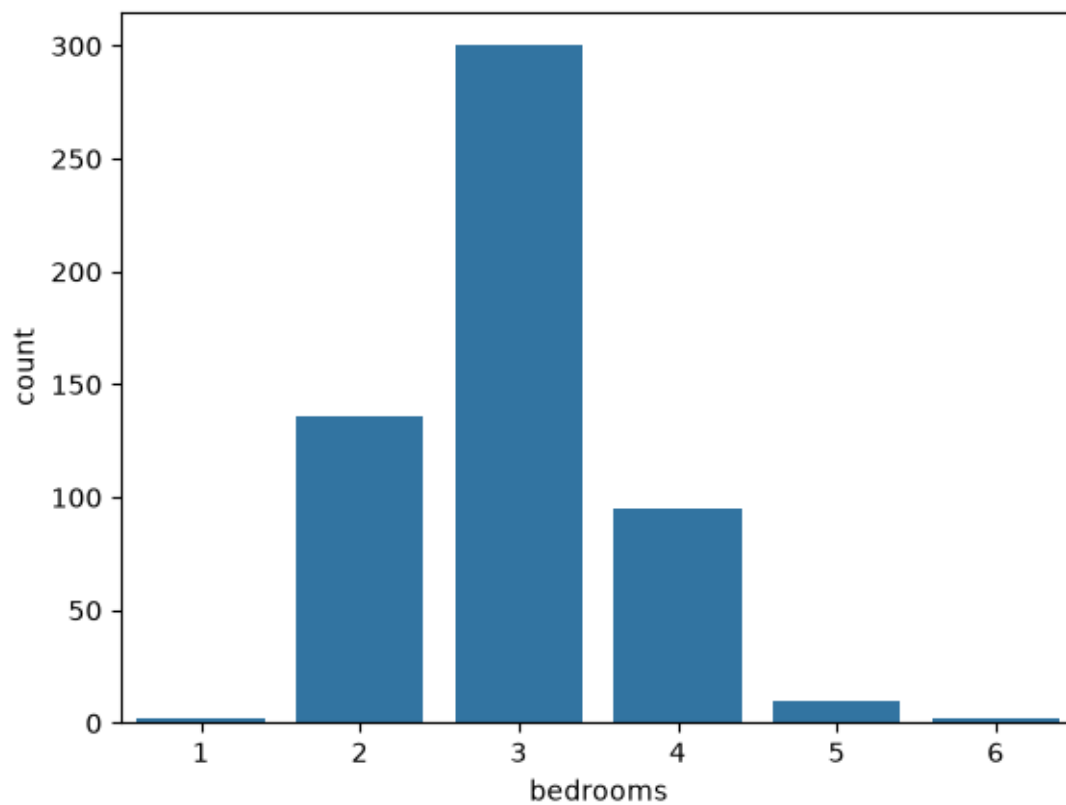
```
[29]: sns.histplot(housing_features['price'], kde=True)
```

```
[29]: <Axes: xlabel='price', ylabel='Count'>
```



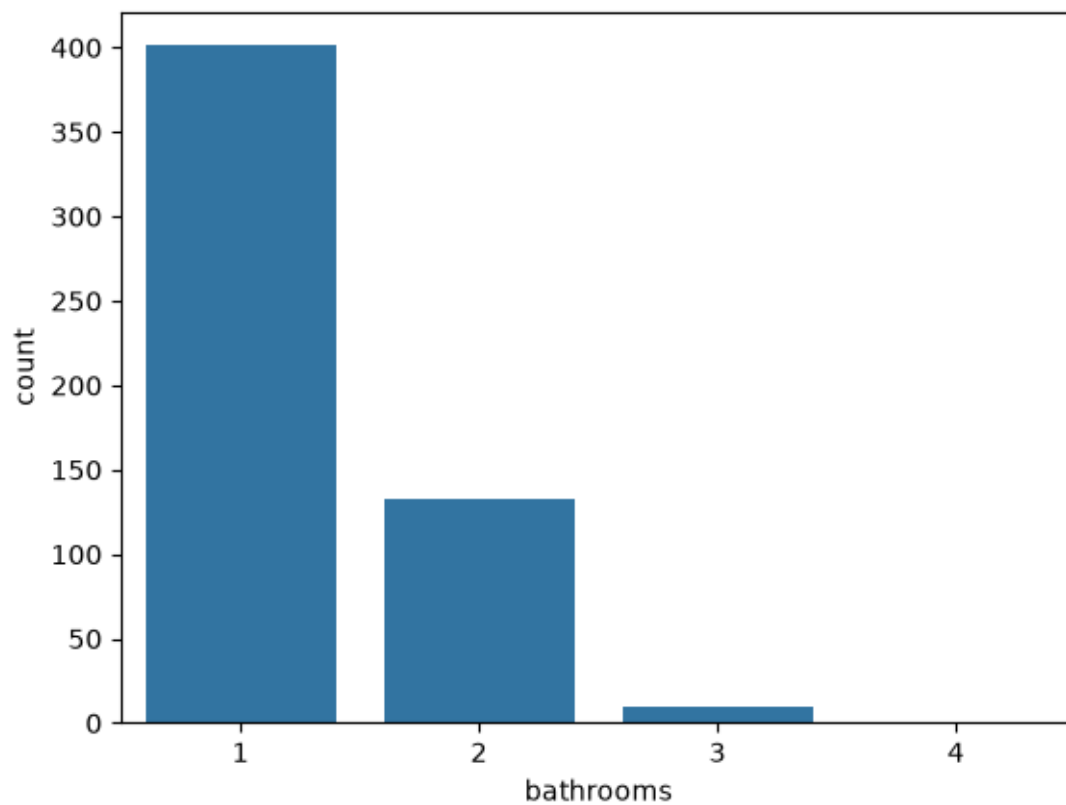
```
[30]: sns.countplot(x='bedrooms', data=housing_features)
```

```
[30]: <Axes: xlabel='bedrooms', ylabel='count'>
```



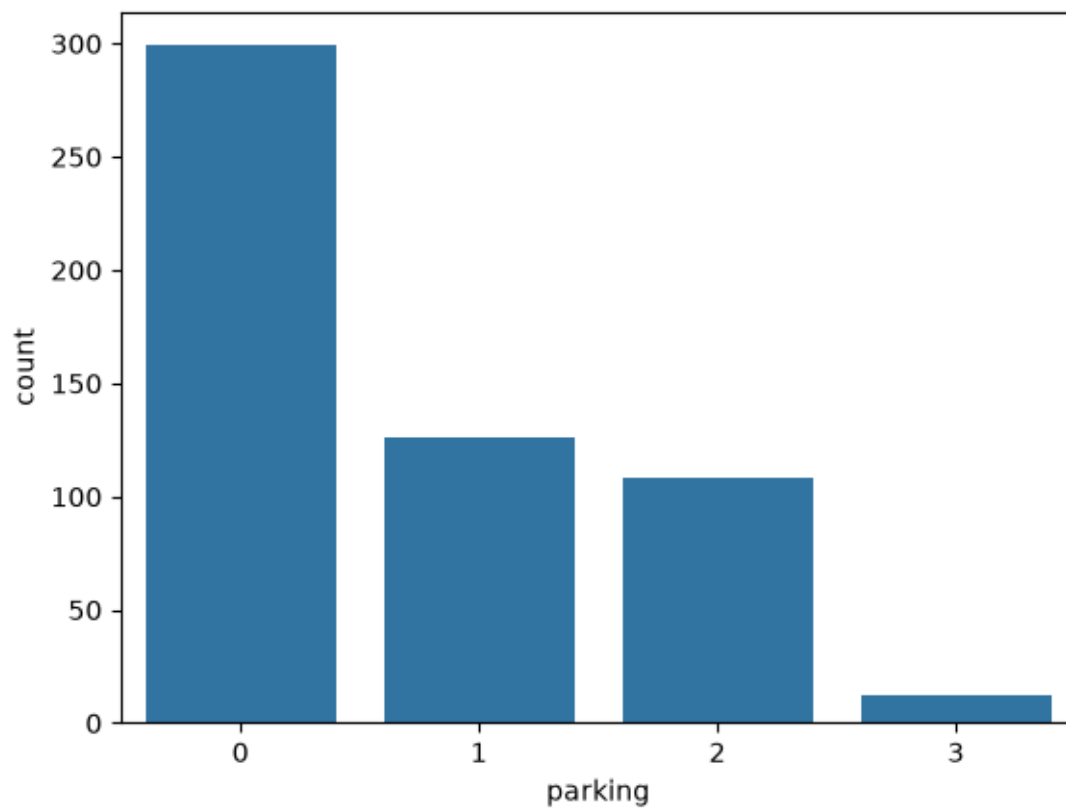
```
[31]: sns.countplot(x='bathrooms', data=housing_features)
```

```
[31]: <Axes: xlabel='bathrooms', ylabel='count'>
```



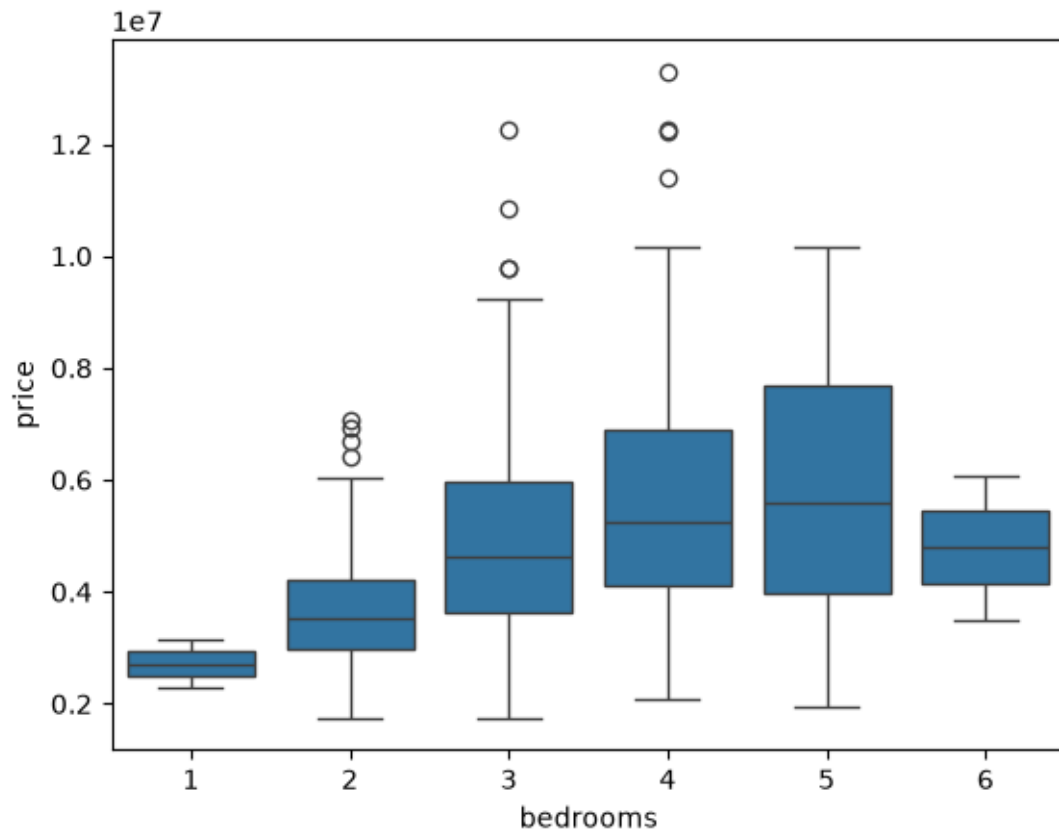
```
[32]: sns.countplot(x='parking', data=housing_features)
```

```
[32]: <Axes: xlabel='parking', ylabel='count'>
```



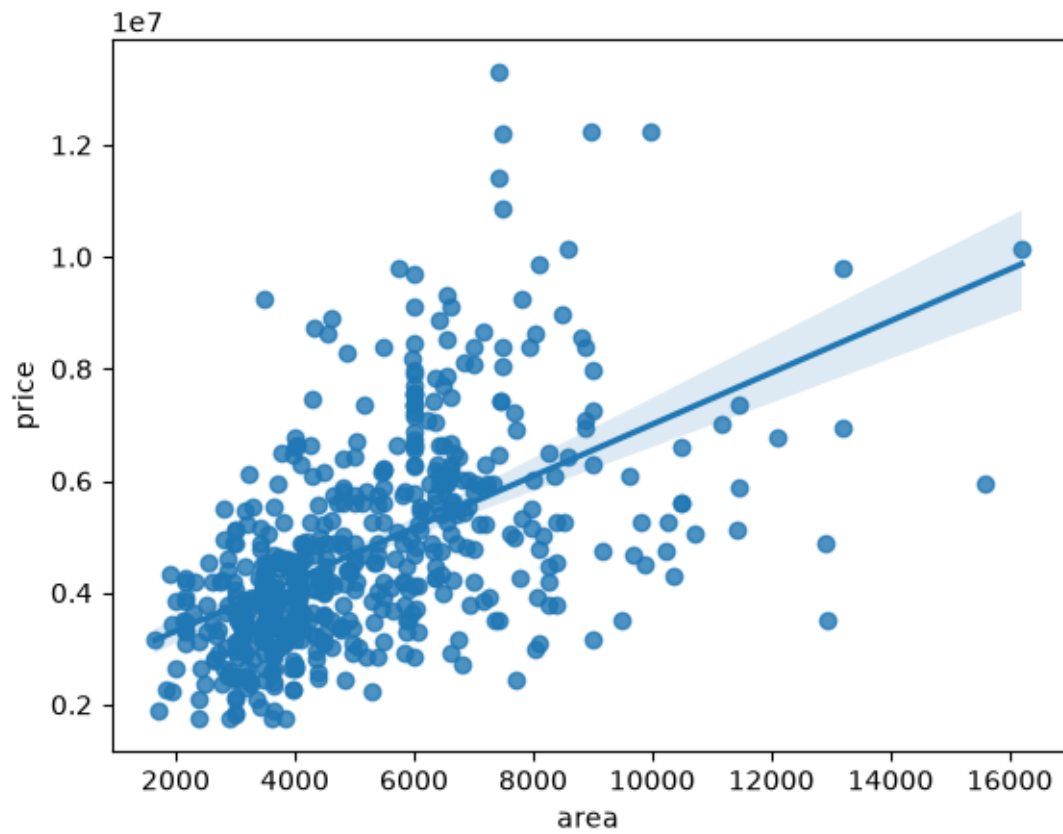
```
[33]: sns.boxplot(x='bedrooms', y='price', data=housing_features)
```

```
[33]: <Axes: xlabel='bedrooms', ylabel='price'>
```



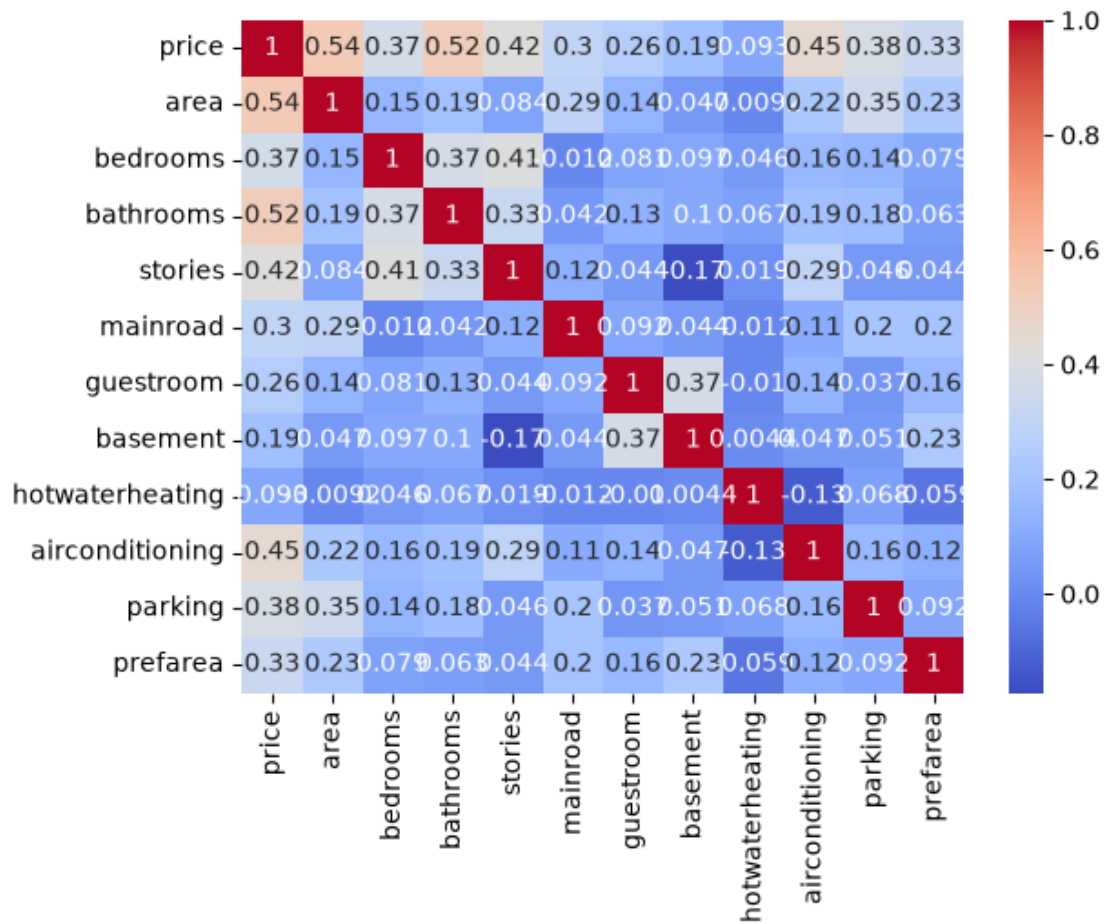
```
[34]: sns.regplot(x='area', y='price', data=housing_features)
```

```
[34]: <Axes: xlabel='area', ylabel='price'>
```



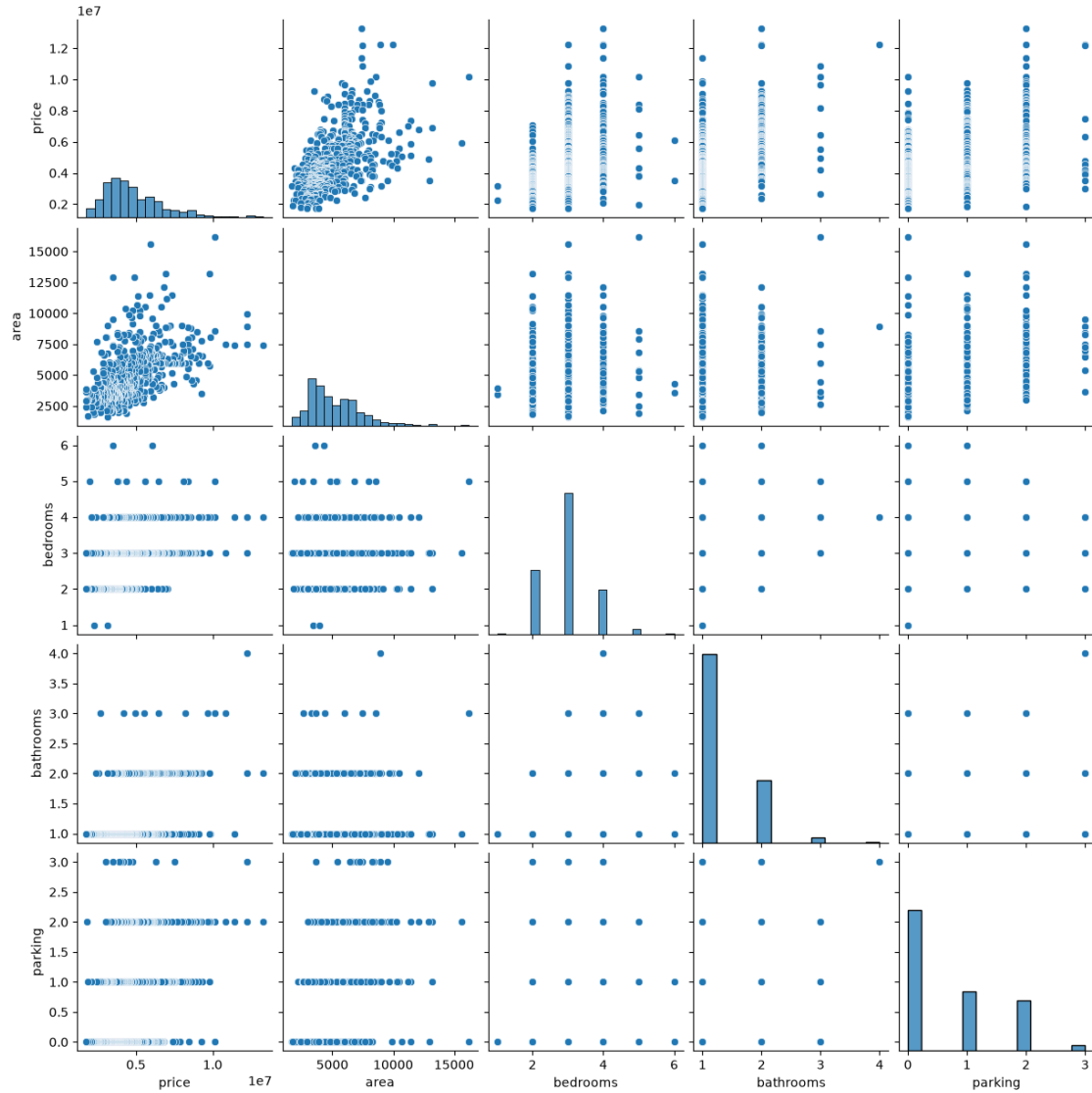
```
[35]: sns.heatmap(housing_features.corr(), annot=True, cmap='coolwarm')
```

```
[35]: <Axes: >
```



```
[36]: sns.pairplot(housing_features[['price', 'area', 'bedrooms', 'bathrooms', 'parking']])
```

```
[36]: <seaborn.axisgrid.PairGrid at 0x27a51be0d70>
```



0.1.4 Logistic Regression Second File

```
[37]: # Create target variable
target2 = "price"
```

```
[38]: # Create data frame without fields model can't evaluate
keep_cols2 = [
    'area',
    'bedrooms',
    'bathrooms',
    'stories',
    'mainroad',
    'guestroom',
```

```

        'basement',
        'hotwaterheating',
        'airconditioning',
        'parking',
        'prefarea']
X2 = housing_features[keep_cols2].copy()
y2 = housing_features[target2].copy()

```

```

[39]: #Split train and test set
X_train2, X_test2, y_train2, y_test2 = train_test_split(X2, y2, test_size=0.2,
↳ random_state=42)

```

```

[40]: # Define scaler and scale continous variables
scaler = StandardScaler()
X_train_scaled2 = scaler.fit_transform(X_train2)
X_test_scaled2 = scaler.transform(X_test2)

```

```

[41]: # Linear Regression Model
lin_model = LinearRegression()
lin_model.fit(X_train_scaled2, y_train2)
lin_pred2 = lin_model.predict(X_test_scaled2)
lin_r2_2 = r2_score(y_test2, lin_pred2)
lin_rmse_2 = mean_squared_error(y_test2, lin_pred2)
print("Linear Regression R²:", lin_r2_2)
print("Linear Regression RMSE:", lin_rmse_2)

```

Linear Regression R²: 0.6437296086614108
Linear Regression RMSE: 1800793719718.9714

```

[42]: mae2 = mean_absolute_error(y_test2, lin_pred2)
mape2 = (mae2 / y_test2.mean()) * 100
print("MAE:", mae2)
print("MAPE:", mape2)

```

MAE: 979112.2162187381
MAPE: 19.552771660303314